

# PIIPS — Manual UAT Server Deployment Guide

Precision Intelligence Invoice Processing Suite — v2.1. FastAPI backend + React frontend, hosted under IIS as a reverse proxy to a Windows-service-hosted Uvicorn process.

*Revised 2026-08-13 — adds PowerShell-based ARR proxy enablement, IIS site port rebinding, ODBC Driver 17 install (with an offline-server fallback), and a PowerShell-based db\_connection fix.*

## 1. Overview

The application runs as two pieces on one Windows server:

- **Backend** — a FastAPI app (`app.py`, exposed as `main:app`) run by Uvicorn, installed as a Windows Service via NSSM so it starts automatically and restarts on crash. It listens on `127.0.0.1:8000` (localhost only — not exposed directly).
- **IIS site** — a plain reverse proxy (URL Rewrite + Application Request Routing) that listens on the public port and forwards every request to the backend on port 8000. The built React frontend is also served from the same backend process, so IIS only ever talks to port 8000.

This guide reproduces that exact setup on a new UAT server, step by step. A companion file, `Deploy_To_Network.txt` (project root), contains every command below as copy-paste-ready PowerShell blocks in the order you run them.

## 2. Prerequisites to install on the UAT server

| Component                                                | Purpose                                  | Notes                                                                                                                                                                                                      |
|----------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python 3.10.x (64-bit)                                   | Runs the backend                         | Match the dev version (3.10.10) to avoid wheel-compatibility issues with <code>paddlepaddle/opencv</code> .                                                                                                |
| Node.js (LTS)                                            | Builds the React frontend                | Only needed if you build on the UAT server itself; skip if you copy a pre-built <code>frontend/dist</code> from a machine that already has Node.                                                           |
| ODBC Driver 17 for SQL Server                            | Backend→SQL Server connectivity (pyodbc) | Native driver, not installed via pip. Download from Microsoft and install with default options. If the UAT server has no internet access, download it on another machine and copy it over first — see §11. |
| Poppler for Windows                                      | PDF→image rasterisation (pdf2image)      | Unzip anywhere (e.g. <code>C:\Poppler</code> ) and add its <code>Library\bin</code> folder to the machine's <b>System</b> PATH environment variable (not just the user PATH).                              |
| IIS with URL Rewrite + Application Request Routing (ARR) | Reverse proxy to the backend             | Installers are bundled in the repo:<br><code>required_softwares\rewrite_amd64_en-US.msi</code> and <code>required_softwares\RequestRouter_amd64.msi</code> .                                               |
| NSSM (Non-Sucking Service Manager)                       | Runs Uvicorn as a Windows Service        | Bundled in the repo:<br><code>required_softwares\nssm-2.24\win64\nssm.exe</code> . No installation needed — it's a standalone exe.                                                                         |

Install IIS itself first (Server Manager → Add Roles and Features → Web Server (IIS)) if it isn't already present, before installing URL Rewrite/ARR on top of it.

### 3. Copy the application to the UAT server

Copy the entire project folder to the UAT server, e.g. to `D:\PIIPS_App`. The project root's `Deploy_To_Network.txt` has a ready-to-edit `robocopy` block that mirrors the source folder to a network destination, excluding the items below automatically. **Do not copy** these folders/files — they are machine-specific or regenerable:

- `venv\` — rebuild fresh on the UAT server (a `venv` is not portable between machines).
- `__pycache__\`, `logs\` — regenerated automatically.
- `config.json` — see §6, the database connection string inside it is encrypted for the *source* machine only and will not decrypt on the UAT server.

**Do** copy, if you want to carry over existing data: `format_model.json` and `model_backups\` (the trained invoice-format recognition model — skip if UAT should start untrained), and `logo\` (static branding assets).

### 4. Set up the Python environment

Open PowerShell as Administrator, in the project folder on the UAT server:

```
cd /d D:\PIIPS_App
python -m venv venv
venv\Scripts\pip install --upgrade pip
venv\Scripts\pip install --no-deps -r requirements.txt
```

`requirements.txt` is a full `pip freeze` dump captured from a working dev machine, so every package is already pinned to a version known to work together. `--no-deps` is required: without it, `pip`'s resolver re-checks each package's own declared dependency ranges and fails with `ResolutionImpossible`, because `paddleocr` declares `opencv-python<=4.6.0.66`, which conflicts with the pinned `opencv-python==4.8.1.78` even though that exact combination is what the dev machine actually runs.

Verify the interpreter version and that the app imports cleanly before continuing:

```
venv\Scripts\python.exe --version
venv\Scripts\python.exe -c "import app; print('OK')"
```

If `python.exe --version` isn't 3.10.x, or if package installs fail with "no matching distribution" (not a resolver conflict, but no wheel found at all for `opencv-python/paddlepaddle`), the `venv` was created with the wrong Python version — delete it and recreate with a 3.10.x interpreter.

### 5. Build the frontend

```
cd frontend
npm install
```

```
npm run build
```

This produces `frontend\dist`, which the FastAPI backend serves directly — no separate web server or copy step is needed for the frontend itself.

## 6. Configure the database connection

The connection string in `config.json` is encrypted at rest using Windows DPAPI at the local-machine scope. A `config.json` copied from another machine will have a connection string that **cannot be decrypted** on the UAT server — the app will treat the database as unconfigured, and login itself will fail with `Database error: db_connection is not configured (server/database missing)` since login needs the database.

Because login needs the database, the UI-based fix (open Database Configuration after logging in) isn't available yet on a fresh UAT server. Set the connection string directly in `config.json` with PowerShell instead (safer than a text editor — avoids encoding or quoting mistakes):

```
$configPath = "D:\PIIPS_App\config.json"
$config = Get-Content $configPath -Raw | ConvertFrom-Json
$config.db_connection = "Server=<host>;Database=<db>;User Id=<user>;Password=<password>;"
$config | ConvertTo-Json | Set-Content $configPath -Encoding utf8
```

Restart the backend service (see §7) so it picks up the change. On startup, the app detects the plaintext value and DPAPI-encrypts it in place using **this** machine's key — seeing `config.json` show `"enc:dpapi:..."` again after a restart is expected and correct, not a sign the fix failed.

Also set `folder_path` (the base folder for Input/Output/status folders, e.g. `D:\PIIPS_Data` — must exist on the UAT server itself) and, if Service First integration is needed, `sf_api_url`. Both can also be set later from the Folder Configuration / API Configuration screens once login succeeds.

## 7. Install the backend as a Windows Service (NSSM)

```
cd /d D:\PIIPS_App\required_softwares\nssm-2.24\win64

nssm.exe install PIIPS_Backend "D:\PIIPS_App\venv\Scripts\python.exe" "-m uvicorn main:app --host 127.0.0.1 --port 8000"
nssm.exe set PIIPS_Backend AppDirectory "D:\PIIPS_App"
nssm.exe set PIIPS_Backend Start SERVICE_AUTO_START
nssm.exe set PIIPS_Backend AppStdout "D:\PIIPS_App\logs\service_stdout.log"
nssm.exe set PIIPS_Backend AppStderr "D:\PIIPS_App\logs\service_stderr.log"
nssm.exe set PIIPS_Backend AppExit Default Restart

net start PIIPS_Backend
```

Confirm it's running and healthy:

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

Expected response: `{"status": "Healthy"}`. The first start can take 15–30 seconds while PaddleOCR initialises

— retry the health check a few times before assuming a failure.

## 8. PaddleOCR model cache (important — avoid on first run)

NSSM services run as **Local System** by default, a different Windows account from whoever is logged in interactively. PaddleOCR downloads its recognition models to the current user's profile (`%UserProfile%\paddleocr`) the first time it runs — under Local System that's `C:\Windows\System32\config\systemprofile\paddleocr`, which is empty on a fresh server. If the UAT server has no internet access, the first OCR call will fail trying to download the models.

Two ways to avoid this:

1. **Pre-seed the cache** — on a machine that already has the models (e.g. the dev machine), copy `%UserProfile%\paddleocr` to `C:\Windows\System32\config\systemprofile\paddleocr` on the UAT server before first start.
2. **Let it download once** — if the UAT server does have outbound internet access, simply let the first request download the models (a one-time delay), then no further action is needed.

## 9. Configure the IIS reverse-proxy site

**Shared UAT server note:** if the UAT server already hosts another site under IIS, PIIPS must be given its own, currently-unused port — it must not reuse that application's port or its Default Web Site binding. PIIPS's app pool runs **No Managed Code** (it only proxies to Uvicorn) and the NSSM-hosted backend listens on `127.0.0.1:8000`, a port private to PIIPS, so both applications otherwise coexist without conflict.

### 9.1 Check what ports are already bound

```
Import-Module WebAdministration
Get-Website | Select-Object Name, State,
    @{n='Bindings';e={(($_.bindings.Collection | ForEach-Object { $_.bindingInformation }) -
join '; '}}
netstat -ano | findstr LISTENING
```

Pick a port for PIIPS that appears in neither list. If `New-Website` later fails with "used by another process" even though the port didn't show as LISTENING, see §11 — it can also be a Windows-reserved dynamic port-exclusion range.

### 9.2 Enable ARR proxying (one-time, server-wide)

Without this step, the reverse-proxy rule in §9.4 will fail with **HTTP 404.4** (`StaticFile` handler, error code `0x8007007b`) — IIS treats the rewrite target as a literal file path instead of proxying to it. Enable it from PowerShell (no-op, and safe to re-run, if another site on this server already proxies via ARR):

```
& "$env:windir\System32\inetsrv\appcmd.exe" set config -section:system.webServer/proxy /
enabled:"True" /commit:apphost
iisreset
```

If `appcmd` errors with "Unrecognized configuration section system.webServer/proxy", ARR itself isn't installed — run `required_softwares\requestRouter_amd64.msi` first, then rerun the two lines above. GUI equivalent: IIS

Manager → server node (top level) → Application Request Routing Cache → Server Proxy Settings → check **Enable proxy** → Apply.

### 9.3 Create the site

```
New-WebAppPool -Name "PIIPS_AI" -Force
Set-ItemProperty "IIS:\AppPools\PIIPS_AI" -Name managedRuntimeVersion -Value ""

New-Website -Name "PIIPS_AI" `
  -PhysicalPath "D:\PIIPS_App" `
  -Port 8080 `
  -ApplicationPool "PIIPS_AI" -Force
```

The application pool's `managedRuntimeVersion` is set to an empty string (No Managed Code) because no managed code runs in this pool; it only proxies. This is a separate app pool from any other application's, so neither one's runtime/recycle settings affect the other.

### 9.4 Reverse-proxy rule

The physical path already contains a `web.config` with the rewrite rule below. If deploying to a folder without it, create `web.config` in the site's physical path with exactly this content:

```
<?xml version='1.0' encoding='utf-8'?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="ReverseProxy" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:8000/{R:1}" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

The target `http://127.0.0.1:8000` must match the `--port` the NSSM service was set up with in §7.

### 9.5 Changing the site's port later

If the site already exists on the wrong port (e.g. it was created on 5000, but 5000 turned out to already be in use), rebind it instead of recreating it:

```
Set-WebBinding -Name "PIIPS_AI" -BindingInformation "*:5000:" -PropertyName "Port" -Value 8080
Get-WebBinding -Name "PIIPS_AI"
Restart-WebItem "IIS:\Sites\PIIPS_AI"
```

## 10. Verification / smoke test

1. Confirm the service is running: `Get-Service PIIPS_Backend` should show **Running**.
2. Confirm the backend answers directly: `Invoke-RestMethod http://127.0.0.1:8000/health` →

```
{"status": "Healthy"}
```

3. Confirm the IIS site is started: `Get-Website -Name "PIIPS_AI"` should show **Started**.
4. Confirm the proxy end-to-end: `Invoke-WebRequest http://localhost:8080/health -UseBasicParsing` → HTTP 200, same JSON body.
5. Open `http://<uat-server>:8080` in a browser, log in, and confirm the Dashboard loads.
6. Go to Folder Configuration and confirm/set the base folder path — the app creates the Input, New\_Format, Trained\_format, Output and per-status subfolders automatically the first time it's saved.
7. Go to Database Configuration and confirm the connection test succeeds (see §6 if it was skipped earlier).
8. Process one sample PDF end-to-end (Dashboard → Start) and confirm it completes without errors.

## 11. Common problems

| Symptom                                                                        | Cause                                                                                                                    | Fix                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IIS shows 502.3 Bad Gateway right after a service restart                      | Uvicorn/<br>PaddleOCR is still initialising (10–30s)                                                                     | Wait and retry; not a real failure if it clears within a minute.                                                                                                                                                           |
| IIS shows 502.3 Bad Gateway persistently                                       | ARR proxy not enabled, or backend not listening on the port web.config targets                                           | Re-check §9.2 and that <code>Invoke-RestMethod http://127.0.0.1:8000/health</code> works directly.                                                                                                                         |
| HTTP 404.4 (Handler: StaticFile, error code 0x8007007b) on the site's own port | ARR proxy not enabled — the rewrite target is treated as a literal file path instead of being proxied                    | Run the §9.2 <code>appcmd + iisreset</code> commands.                                                                                                                                                                      |
| <code>New-Website</code> fails: "...used by another process"                   | The chosen port is already bound — sometimes by a Windows dynamic port-exclusion range with nothing showing as LISTENING | <code>netstat -ano   findstr :&lt;port&gt;</code> then <code>Get-Process -Id &lt;pid&gt;</code> ; also check <code>netsh interface ipv4 show excludedportrange protocol=tcp</code> . Pick a free port, or rebind per §9.5. |
| Database Configuration / login shows "db_connection is not configured"         | DPAPI-encrypted connection string from a different machine, or never set                                                 | Re-enter it fresh on this server via the PowerShell method in §6.                                                                                                                                                          |

|                                                                                                          |                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Database error: [IM002]<br>Data source name not found and no default driver specified (SQLDriverConnect) | ODBC Driver 17 for SQL Server not installed                                   | <p>Check with <code>Get-OdbcDriver -Name "ODBC Driver 17 for SQL Server"</code>. If missing, and this server has internet access:</p> <pre>Invoke-WebRequest -Uri "https://go.microsoft.com/fwlink/?linkid=2200732" -OutFile "\$env:TEMP\msodbcsql17.msi" Start-Process msixec.exe -ArgumentList "/i `"\$env:TEMP\msodbcsql17.msi`" /passive /norestart IACCEPTMSODBCSQLLICENSETERMS=YES" -Wait -PassThru   Select-Object ExitCode</pre> <p>If the server has <b>no</b> internet access (<code>Invoke-WebRequest</code> fails with "Unable to connect to the remote server"), download the MSI on another machine, copy it over (confirm reachability first with <code>Test-Connection</code> / <code>Test-NetConnection -Port 445</code>; use <code>New-PSDrive ... -Credential (Get-Credential)</code> if the plain admin share copy fails, or RDP file transfer as a last resort), then run the same <code>msixec</code> command against the local copy. ExitCode 0 = success.</p> |
| First PDF processed fails with a network/DNS error mentioning paddleocr.bj.bcebos.com                    | Local System's PaddleOCR cache is empty and the server has no internet access | Pre-seed the cache as described in §8.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| PDF processing fails with a poppler/pdfinfo error                                                        | Poppler not on the System PATH, or only on the interactive user's PATH        | Add Poppler's <code>Library\bin</code> to the <b>System</b> PATH (not User PATH) and restart the PIIPS_Backend service so it picks up the new PATH.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

## Appendix — quick command reference

```
# Check/restart the backend service
Get-Service PIIPS_Backend
Restart-Service PIIPS_Backend

# Check/restart the IIS site
Get-Website -Name "PIIPS_AI"
Restart-WebItem "IIS:\Sites\PIIPS_AI"

# Rebind the site to a different port
Set-WebBinding -Name "PIIPS_AI" -BindingInformation "*:<old-port>:" -PropertyName "Port" -
Value <new-port>

# Enable ARR proxying (fixes persistent 502.3 / 404.4)
& "$env:windir\System32\inetsrv\apppcmd.exe" set config -section:system.webServer/proxy /
enabled:"True" /commit:apphost
iisreset

# Health checks
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-WebRequest http://localhost:8080/health -UseBasicParsing
```

```
# ODBC driver check
Get-OdbcDriver -Name "ODBC Driver 17 for SQL Server"

# Service logs
Get-Content D:\PIIPS_App\logs\service_stdout.log -Tail 50
Get-Content D:\PIIPS_App\logs\service_stderr.log -Tail 50
```

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|